数据库系统实验报告

实验 01：存储管理

MiniDB ——段页式存储、空闲空间管理与缓冲区管理

课程名称： 数据库系统

小组成员： 任勇宁，彭文博，边宇晗，章国文，张睿涵

提交日期： 2026 年 10 月 28 日

代码仓库：https://github.com/renyongning/MiniDB

目录

1 实验分工安排 **2**

2 实验环境 **3**

3 实验设计与实现 **3**

3.1 设计目标 . . . 3
3.2 总体架构 . . . 3
3.3 数据文件组织 . . . 4
3.3.1 整体布局：段页式，每个关系一组文件 . . . 4
3.3.2 通用页头 . . . 5
3.3.3 文件头页（页 0）与区位图页（页 1） . . . 5
3.3.4 段头页 . . . 5
3.3.5 堆页（slotted page） . . . 6
3.3.6 类型系统与记录格式 . . . 6
3.4 空闲空间管理 . . . 7
3.4.1 三层结构 . . . 7
3.4.2 区的分配与回收（SpaceManager） . . . 7
3.4.3 页的分配与回收（Segment） . . . 7
3.4.4 记录级空间：FSM . . . 7
3.5 缓冲区管理 . . . 8
3.5.1 数据结构 . . . 8
3.5.2 查找与读入 . . . 8
3.5.3 淘汰策略：CLOCK . . . 8
3.5.4 PageGuard . . . 9
3.5.5 内外存数据交换 . . . 9
3.6 记录的增删改 . . . 9
3.7 系统目录 . . . 9
3.8 可观测性：日志与 PRAGMA . . . 10
3.9 模块接口 . . . 10
3.9.1 对外 C API（`include/minidb/minidb.h`） . . . 10
3.9.2 存储层内部接口 . . . 10

4 使用手册 **11**

4.1 编译 . . . 11
4.2 启动 shell . . . 12
4.3 SQL . . . 12
4.4 元命令 . . . 13
4.5 导入 TPC-H 数据与演示 . . . 13

5 测试方案和结果 **14**
5.1 测试方案 14
5.2 自动化测试结果 14
5.3 supplier 演示结果 14
5.4 规模测试：TPC-H SF1 16
5.5 测试中发现的问题与重点覆盖的边界情况 17

6 实验总结 **18**
6.1 完成情况 18
6.2 收获与体会 18
6.3 不足与后续工作 19

A 核心源代码 **19**
A.1 CLOCK 淘汰（`src/buffer/buffer_pool.cpp`） 19
A.2 缓冲区查找与换入（`src/buffer/buffer_pool.cpp`，有删节） 19
A.3 区分配（`src/space/space_manager.cpp`） 21
A.4 页分配与回收（`src/space/segment.cpp`，有删节） 21
A.5 FSM 搜索（`src/space/segment.cpp`，有删节） 22
A.6 记录更新与转发指针（`src/table/table_heap.cpp`，有删节） 23

1 实验分工安排

课程共五次实验，最终目标是一个可运行的小型关系数据库。实验一完成存储层：数据文件组织、空闲空间管理和缓冲区管理，并为后续的索引、查询处理、并发控制和故障恢复实验预留接口与磁盘格式字段。

表 1: 小组分工

成员	负责内容
任勇宁	总体设计与设计文档；DiskManager、页格式、CRC32C 校验和
彭文博	BufferPool：帧与页表、CLOCK 淘汰、PageGuard、缓冲区统计
边宇晗	SpaceManager（区位图）、Segment（区列表与页掩码）、FSM；TableHeap 与转发指针
章国文	类型系统与记录编解码、Catalog、SQL 解析、C API、shell、COPY 导入、PRAGMA
张睿涵	测试（存储层、集成、SF1 规模测试与逐行校验）、演示脚本、实验报告

进度安排

- 10/22：设计评审，冻结磁盘格式（设计文档 v1.0）；
- 10/23：工程骨架、DiskManager、通用页头、BufferPool、CLOCK、PageGuard；
- 10/24：类型系统、记录编解码、堆页（slotted page）；
- 10/25：SpaceManager、Segment、FSM；
- 10/26：TableHeap、Catalog、shell、批量导入，supplier 端到端打通；
- 10/27：SF1 规模测试与逐行校验、修复缺陷、PRAGMA 与演示脚本；
- 10/28：实验报告、打包提交；10/30：验收。

2 实验环境

表 2: 开发与测试环境

项目	说明
编程语言	C++20（不使用 C++23 特性和 modules）
构建工具	CMake ≥ 3.16，提供 `CMakePresets.json`（`dev`：Debug + ASan/UBSan；`release`：O2）
第三方依赖	无。CRC32C、日志、SQL 解析、测试断言均为自行实现
主平台	Linux x86-64（GCC ≥ 11 / Clang ≥ 14）；Windows 下通过 WSL 构建运行
已验证平台	macOS 27（Apple M4 Pro，arm64），Apple clang 17.0.0，CMake 4.4.4
验收平台	【待填写：验收时使用的操作系统与编译器版本】

与平台相关的代码只有两处，均有可移植的回退路径：

- 文件扩展：Linux 上用 `fallocate` 预留真实磁盘块（文件不稀疏，磁盘满时在扩展时就报错），其他平台（如 macOS）或不支持的文件系统回退到 `ftruncate`；
- CRC32C 校验和：x86-64 上运行时探测 SSE4.2，支持时使用 `crc32` 指令，否则（含 ARM）使用查表实现。

代码规模：核心库、shell 与测试合计约 9200 行 C++。

3 实验设计与实现

3.1 设计目标

编号	目标	说明
G1	磁盘格式一次定型	后续实验需要的页头字段（`page_lsn`）、记录头字段（`txn_id`、`undo_ptr`）在实验一就预留
G2	接口稳定、实现可替换	淘汰策略、FSM 搜索、日志管理器都通过接口隔离
G3	规模可扩展	单个关系最大约 31.8 GB，可完整导入 TPC-H SF1
G4	过程可观测	缓冲区命中/淘汰/刷盘、区和页的分配/回收都有结构化日志与计数器
G5	易于验收	一条命令构建，无第三方依赖

3.2 总体架构

系统按层次组织，上层只依赖下层；除 BufferPool 和 DiskManager 外，任何模块都不直接读写磁盘。所有页访问都通过缓冲池提供的 PageGuard 完成。

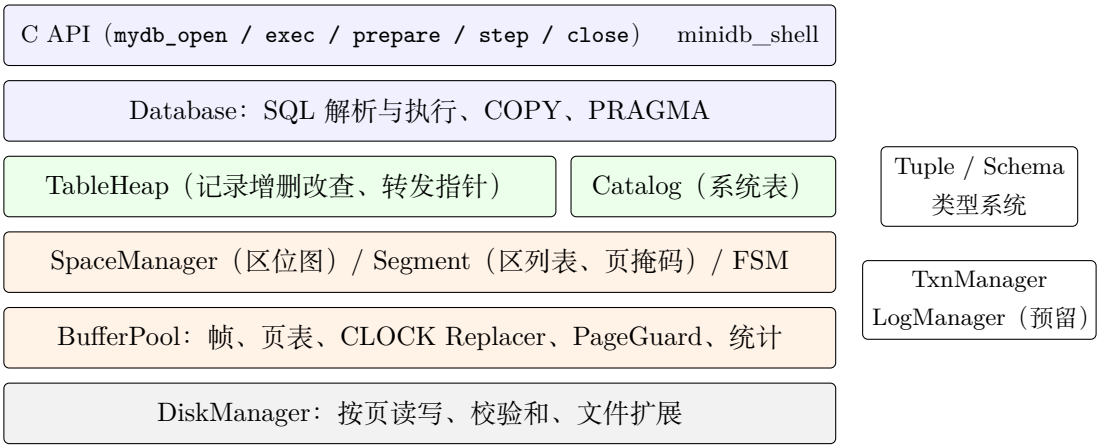


图 1: MiniDB 分层结构

表 3: 模块职责

模块	职责	源文件
DiskManager	按页号读写数据文件；扩展文件；计算/校验页校验和	`src/storage/`
BufferPool	页缓存、查找、淘汰、刷盘；PageGuard 与页锁存	`src/buffer/`
SpaceManager	区的分配与回收（区位图）；段的创建	`src/space/`
Segment	段内页的分配与回收；FSM 搜索与更新	`src/space/`
FreeSpaceMap	每个数据页一字节的空闲空间类别，存于独立 `.fsm` 文件	`src/space/`
TableHeap	记录的增删改查与扫描；转发指针；页内整理	`src/table/`
Tuple/Schema	类型系统、记录编解码	`src/record/`
Catalog	表与列的元数据，持久化在系统表中	`src/catalog/`
Database	组件装配、语句执行（含 COPY、PRAGMA）、打开/关闭流程	`src/db/`

3.3 数据文件组织

3.3.1 整体布局：段页式，每个关系一组文件

数据库是一个目录。每个关系（用户表和两张系统表）对应一个数据文件 `<name>.dat` 和一个空闲空间映射文件 `<name>.fsm`。数据文件内部是段页式组织：

- **页**：8 KB，是 I/O 和缓冲的基本单位；
- **区（extent）**：连续 64 页（512 KB），是文件向段分配空间的单位；
- **段（segment）**：区的集合，段的第一个区的第一页是段头页。目前每个数据文件中有一个堆段，格式允许一个文件有多个段。

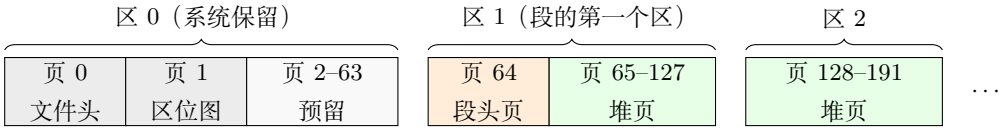


图 2: 数据文件布局（supplier.dat）

页号是文件内页号，文件偏移为 page_id × 8192（64 位运算）；区号到页号为 extent_id × 64。缓冲池用 (file_id, page_id) 标识一页。

选择“每个关系一组文件”而不是单一大文件，参考了 PostgreSQL 和 InnoDB 的 file_per_table：DROP TABLE 可以直接删除文件；区位图只限制单个关系的大小（约 31.8 GB），整个库不受限制；FSM 在独立文件中，恢复实验中可以整体丢弃后按数据页重建。

3.3.2 通用页头

所有页的前 32 字节是通用页头：

偏移	长度	字段	说明
0	4	checksum	CRC32C，计算时该字段视为 0
4	2	page_type	文件头/区位图/段头/区列表溢出/FSM/堆页/B+ 树（预留）
6	2	flags	预留
8	4	page_id	页自身编号，读入时校验
12	4	segment_id	所属段
16	8	page_lsn	最后修改本页的日志 LSN（实验一恒为 0，供恢复实验使用）
24	8	—	预留

DiskManager 写页时对页的副本计算校验和后写出，不修改缓冲帧；读页后校验校验和与 page_id，可以发现磁盘损坏和读错页。

3.3.3 文件头页（页 0）与区位图页（页 1）

文件头页记录魔数 "MINIDBF2"、格式版本、页大小、区大小、文件当前区数、文件类型（数据/FSM）、所属表号、段头页号和 clean_shutdown 标志。打开文件时逐项校验。

区位图页从偏移 40 开始是 8152 字节的位图，每一位对应一个区（1 = 已分配），共 65216 位，可管理 $65216 \times 512\,\text{KB} \approx 31.8\,\text{GB}$。偏移 32 处的 alloc_hint 记录下次分配的起始位置。

3.3.4 段头页

段头页包含 192 字节的段元信息（段号、类型、区列表槽位数、区数、页数、溢出页链表头、插入提示）和一个区列表。区列表每项 16 字节：

字段	长度	说明
`extent_id`	4	区号，无效值表示空洞
`reserved`	4	
`used_mask`	8	64 位页掩码，第 i 位为 1 表示区内第 i 页已分配

段头页内可放 $(8160 - 192)/16 = 498$ 项（约 249 MB），超出的部分放入从段自身分配的溢出页（每页 509 项，链表连接）。TPC-H 的 lineitem 表有 1739 个区，走了溢出页路径。

段内页号 `spno = 区列表下标 × 64 + 区内偏移`，FSM 以 spno 为下标。为保证 spno 稳定，区被回收时在区列表中留下空洞，以后新分配的区优先填入空洞，列表不做压缩。

3.3.5 堆页（slotted page）

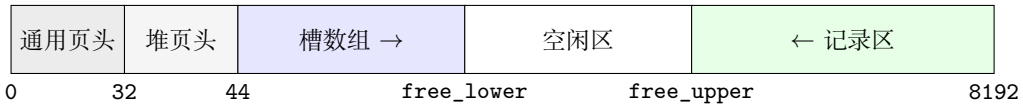


图 3: 堆页布局

堆页头 12 字节：`slot_count`、`free_lower`、`free_upper`、`frag_bytes`（删除和缩短产生的碎片）、`live_count`。每个槽 4 字节：16 位偏移、13 位长度、3 位标志（`USED`、`FORWARD`、`MOVED_IN`），标志为 0 表示空槽，可以复用。

页的可用空间 = 连续空闲（`free_upper - free_lower`）+ 碎片。插入时如果连续空闲不够而总可用够，先做页内整理：把有效记录依次搬到页尾，槽号不变，因此 RID 不变。

3.3.6 类型系统与记录格式

表 4: 类型映射（覆盖实验要求的全部基本类型）

SQL 类型	内部类型	长度	说明
`INT` / `INTEGER`	INT32	4	
`LONG` / `BIGINT`	INT64	8	
`FLOAT`	DOUBLE	8	SQL 标准中 FLOAT 默认双精度；TPC-H 价格有 8 位有效数字，单精度会丢失精度
`DOUBLE`	DOUBLE	8	
`DATE`	DATE	4	自 1970-01-01 起的天数
`CHAR(n)`	CHAR	n	定长，空格填充，输出时去掉尾部空格
`VARCHAR(n)`	VARCHAR	变长	写入时检查长度 $\leq n$
`DECIMAL(p,s)`	DECIMAL	8	额外支持：按 scale 定点化的 int64

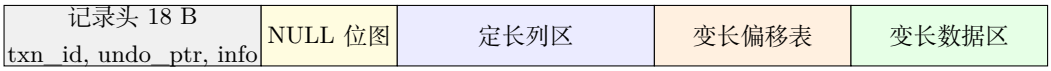


图 4: 记录布局

记录头中的 `txn_id`、`undo_ptr` 为并发控制和恢复实验预留，`info` 的第 0 位是逻辑删除标记。定长列按 schema 顺序紧密排列；每个 VARCHAR 在偏移表中有一个 `uint16` 记录结束偏移，因此读任意一列都是 $O(1)$。字段不做对齐，统一用 `memcpy` 读写，避免未对齐访问的未定义行为。

3.4 空闲空间管理

3.4.1 三层结构

层级	数据结构	分配单位	位置
文件 → 段	区位图	区（512 KB）	数据文件页 1
段 → 页	区列表 + 64 位页掩码	页（8 KB）	段头页及溢出页
页 → 记录	FSM + 页内槽目录	字节	`.fsm` 文件、堆页

3.4.2 区的分配与回收（SpaceManager）

分配：从 `alloc_hint` 开始循环扫描区位图，找到第一个为 0 的位，置 1 并更新 `alloc_hint`；若区号超出文件当前长度，扩展文件并更新文件头。**回收**：清位，若区号小于 `alloc_hint` 则把提示前移，下次分配优先复用。回收已经空闲的区会报错，防止重复回收。

3.4.3 页的分配与回收（Segment）

分配：

1. 从上次成功分配的区列表下标开始循环查找 `used_mask` 未满的区；
2. 找到后用 `__builtin_ctzll(~mask)` 取最低的 0 位，置位，页数加 1；
3. 所有区都满时向 SpaceManager 申请新区，优先填入区列表的第一个空洞，没有空洞才追加；
4. 段元信息写回段头页；新页由缓冲池直接在内存中初始化，不读磁盘。

回收：清对应位，页数减 1，FSM 项置 0，从缓冲池丢弃该页的帧；如果该区的页掩码变为 0（且不是段的第一个区），把区还给区位图，区列表项置为空洞。段头页和溢出页不能被回收。

3.4.4 记录级空间：FSM

FSM 文件中每个数据页对应一个字节，值为“可用字节数 / 32”（0–254）。FSM 只是提示：修改不写日志；如果 FSM 显示某页够用而实际不够，插入失败后用实际值修正该项，继续搜索。

插入时找页：

1. 计算所需类别 $\lceil (\text{记录长度} + 4)/32 \rceil$；

2. 从 `insert_hint` 开始扫描 FSM，找第一个类别足够的页；到末尾后回绕，最后扫描起始 FSM 页中 hint 之前的部分（否则 hint 之前释放的空间永远不会被复用）；
3. 内存中为每个 FSM 页维护“最大类别”的上界，上界小于所需类别的 FSM 页整页跳过，大表找不到空闲页时不必每次扫描整个 FSM；
4. 找不到则分配新堆页；插入后用页的实际可用空间更新 FSM。

删除：槽置空，长度计入碎片，更新 FSM；页中有效记录数降到 0 时回收整页（段的第一个堆页保留，避免表在空与非空之间反复变化时频繁分配回收）。

3.5 缓冲区管理

3.5.1 数据结构

- **帧数组**：固定数量的帧（默认 1024 帧 = 8 MB，可用 `--buffer` 调整），每帧包含页数据、所属文件、页号、pin 计数、脏标记和一把读写锁存（`std::shared_mutex`）；
- **页表**：`unordered_map<(file_id, page_id), frame_id>`，$O(1)$ 查找；
- **空闲帧链表**：尚未使用或被丢弃的帧；
- **Replacer**：淘汰策略接口（`RecordAccess`、`SetEvictable`、`Evict`、`Remove`），默认实现为 CLOCK；
- **统计**：hit、miss、evict、flush、new_page 和命中率。

3.5.2 查找与读入

1. 加全局互斥锁，查页表；
2. **命中**：pin 计数加 1，若 pin 从 0 变为 1 则在 Replacer 中标记为不可淘汰，记录访问，hit 加 1；
3. **未命中**：先从空闲链表取帧；没有空闲帧则由 Replacer 选出牺牲帧（只会选 pin 为 0 的帧），所有帧都被 pin 时返回“缓冲池已满”错误。牺牲帧若为脏页先写回磁盘（flush 加 1），然后从页表删除旧映射、插入新映射，从磁盘读入新页并校验。读盘失败时帧放回空闲链表；
4. 释放全局锁后再获取页锁存（读或写）。持有全局锁时不等待页锁存，避免死锁。

3.5.3 淘汰策略：CLOCK

每帧一个引用位，访问时置 1。时钟指针循环扫描：跳过不可淘汰（被 pin）的帧；遇到引用位为 1 的帧清为 0 继续；遇到可淘汰且引用位为 0 的帧则淘汰。最多扫描两圈。CLOCK 近似 LRU，但每次访问只需置一位，不需要维护链表，PostgreSQL 使用的也是这一类算法。Replacer 是接口，可以替换为 LRU-2 等策略而不影响其他模块。

3.5.4 PageGuard

所有模块只通过 `ReadPageGuard`（pin + 共享锁存）和 `WritePageGuard`（pin + 独占锁存）访问页。Guard 采用 RAII：析构时自动释放锁存并 unpin；`WritePageGuard::MutableData()` 自动标记脏页。这样就不会出现“忘记 unpin 导致缓冲池耗尽”或“修改后忘记标脏”的问题。

3.5.5 内外存数据交换

- **换入**：未命中时从磁盘读页，校验校验和与页号；
- **换出**：淘汰脏页时写回；写回在 8 KB 的栈上副本中计算校验和，不修改帧内容；
- **新页**：不读磁盘，直接在帧中清零并初始化页头，只计 new_page；
- **刷盘时机**：淘汰时、关闭数据库时、`.flush`（`PRAGMA flush`）时，`FlushAll` 写回全部脏页后 `fsync`；
- **WAL 钩子**：写回脏页前调用 `LogManager::FlushUntil(page_lsn)`，实验一为空操作，恢复实验时保证 WAL 规则。

3.6 记录的增删改

TableHeap 提供 `Insert`、`Delete`、`Update`、`Get` 和顺序扫描。记录用 RID（页号 + 槽号）标识。

插入 编码记录 → FSM 找页 → 页内插入（优先复用空槽，必要时整理）→ 更新 FSM。

删除 若槽是转发指针，先删目标再删原槽；更新 FSM，页空时回收整页。

更新 先通过原 RID 找到记录当前位置，然后：

1. 新记录不长于原记录：原地覆盖，差值计入碎片；
2. 新记录更长但本页放得下：先把旧记录计入碎片，必要时整理，再写入，槽号不变；
3. 本页放不下：新版本以 `USED|MOVED_IN` 插入到其他页，原槽改为 6 字节的转发指针（目标 RID）。原 RID 保持不变，以后的索引、锁和 undo 都可以继续使用它。

转发链最多一跳：已经被转发的记录再次移动时，直接更新原槽的转发指针并删除旧目标。顺序扫描跳过 `MOVED_IN` 记录，遇到 `FORWARD` 时跟随并以原 RID 输出，保证每条记录恰好输出一次。`UPDATE/DELETE` 先扫描收集匹配的 RID，再逐条修改，避免被移动到后面页的记录在同一次扫描中被再次匹配（Halloween 问题）。

3.7 系统目录

两张系统表 `sys_tables`、`sys_columns` 本身就是普通堆表，schema 硬编码，段头页号记录在各自数据文件的文件头中。打开数据库时扫描系统表重建内存中的表信息，并按其中记录的文件名打开其余关系的文件。元数据存成普通表，恢复实验中不需要额外的持久化机制。

3.8 可观测性：日志与 PRAGMA

日志 格式为 `[时间][级别][模块] 事件 key=value`，模块包括 DISK、BUF、SPACE、FSM、HEAP、CAT、LOAD、DB。DEBUG 级别输出缓冲区未命中、淘汰、新页、刷盘，区和页的分配回收，FSM 类别变化，记录增删改；INFO 级别只输出导入进度与摘要。日志可同时输出到控制台和文件，级别可通过 `PRAGMA log_level` 动态修改。级别关闭时日志宏只做一次比较，不构造消息。

PRAGMA 参考 SQLite 的 PRAGMA 和 PostgreSQL 的 `pg_buffercache`、pageinspect，把引擎内部状态以结果集形式暴露，shell、C API 和测试都能直接使用：

PRAGMA	内容
`buffer_stats`	帧数、已用、pin、脏页、hit/miss/命中率、evict/flush/new_page
`buffer_frames`	每个帧中的文件、页号、页类型、pin 计数、脏标记
`table_info(t)`	文件大小、段头页、区数、页数、行数、转发数、空闲字节、填充率
`extent_bitmap(t)`	数据文件的区位图与 `alloc_hint`
`extents(t)`	区列表与每个区的 64 位页掩码（`H` 段头页，`1` 已分配，`.` 空闲）
`free_space(t)` / `free_space_summary(t)`	每页的 FSM 类别与实际空闲字节 / 空闲空间直方图
`page_header(t,p)` / `page_slots(t,p)`	页头字段 / 槽目录（偏移、长度、标志、转发目标）
`flush`、`log_level`	刷盘；查看或设置日志级别

3.9 模块接口

3.9.1 对外 C API（include/minidb/minidb.h）

接口风格与 SQLite 一致，供 shell、测试以及后续实验的上层模块使用：

```c
int  mydb_open(const char* path, const mydb_config* config, mydb_db** out_db);
int  mydb_close(mydb_db* db);
int  mydb_exec(mydb_db* db, const char* sql, char** errmsg);
int  mydb_prepare(mydb_db* db, const char* sql, mydb_stmt** out_stmt);
int  mydb_step(mydb_stmt* stmt);            /* MYDB_ROW / MYDB_DONE */
int  mydb_column_count(mydb_stmt* stmt);
const char* mydb_column_name(mydb_stmt* stmt, int column);
int  mydb_column_type(mydb_stmt* stmt, int column);
const char* mydb_column_text(mydb_stmt* stmt, int column);
void mydb_finalize(mydb_stmt* stmt);
int  mydb_changes(mydb_db* db);
const char* mydb_errmsg(mydb_db* db);
```

3.9.2 存储层内部接口

```
class DiskManager {
  static Result<Ptr> Create(const std::string& path, FileKind kind, uint32_t table_id);
  static Result<Ptr> Open(const std::string& path, FileKind expected_kind);
  Status ReadPage(page_id_t page_id, std::byte* out);         // verifies checksum + page_id
  Status WritePage(page_id_t page_id, const std::byte* data); // checksums a copy
  Status ExtendTo(uint32_t extents);
  Status Sync();
};
class BufferPool {
  Result<ReadPageGuard>  FetchRead(const std::shared_ptr<DiskManager>&, page_id_t);
  Result<WritePageGuard> FetchWrite(const std::shared_ptr<DiskManager>&, page_id_t);
  Result<WritePageGuard> NewPage(...);
  Status FlushPage(...);  Status FlushAll();
  void   DiscardPage(...);
  const BufferStats& Stats() const;
};
class Replacer {   // CLOCK by default
  virtual void RecordAccess(frame_id_t) = 0;
  virtual void SetEvictable(frame_id_t, bool) = 0;
  virtual std::optional<frame_id_t> Evict() = 0;
  virtual void Remove(frame_id_t) = 0;
};
class SpaceManager {
  Result<extent_id_t> AllocateExtent(Transaction*);
  Status              FreeExtent(Transaction*, extent_id_t);
  Result<std::unique_ptr<Segment>> CreateSegment(Transaction*, ...);
};
class Segment {
  Result<page_id_t> AllocatePage(Transaction*);
  Status            FreePage(Transaction*, page_id_t);
  Result<page_id_t> FindPageWithSpace(uint32_t need_bytes);  // FSM search
  void              UpdateFreeSpace(page_id_t, uint32_t free_bytes);
};
class TableHeap {
  Result<RID>   Insert(Transaction*, const Tuple&);
  Status        Delete(Transaction*, RID);
  Result<RID>   Update(Transaction*, RID, const Tuple&);  // RID never changes
  Result<Tuple> Get(Transaction*, RID);
  Iterator      Scan(Transaction*);
};
```

可预期的失败（记录不存在、空间不足、类型不匹配）通过 `Result<T>` / `Status` 返回；不变量被破坏（校验和错误、页类型不符）由断言报告。

4 使用手册

4.1 编译

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
ctest --test-dir build --output-on-failure      # 运行测试
```

生成的程序：

- `minidb_shell`：交互式命令行；
- `minidb_demo`：嵌入式 C API 使用示例；
- `minidb_tests`、`minidb_storage_tests`：集成测试和存储层测试；
- `minidb_verify_tpch`：把库中每张表与 `.tbl` 原文逐行比对；
- `libminidb_core.a`、`libminidb.so`/`.dylib`：核心库。

4.2 启动 shell

```
./build/minidb_shell --db <数据库目录> [--buffer <帧数>] [--log-level debug|info|warn] [--file <脚
    本>]
```

目录不存在时自动创建新库。`--buffer` 设置缓冲池帧数（每帧 8 KB），演示淘汰时可以调小；`--file` 执行脚本文件。脚本里可以混写 SQL 和元命令，以 `--` 开头的行是注释。

4.3 SQL

```
CREATE TABLE supplier (S_SUPPKEY INTEGER NOT NULL, S_NAME CHAR(25) NOT NULL, ...);
DROP TABLE t;
INSERT INTO supplier VALUES (20001, 'Supplier#000020001', 'addr', 7, '17-123-456-7890', 1234.56,
    'c');
SELECT rid, s_suppkey, s_name FROM supplier WHERE s_suppkey <= 5 LIMIT 10;
SELECT * FROM supplier WHERE rid = '65:0';        -- rid 伪列：页号:槽号
UPDATE supplier SET s_acctbal = 0 WHERE s_suppkey = 1;
DELETE FROM supplier WHERE s_suppkey <= 3000;
COPY supplier FROM 'test/tpc-h/supplier.tbl' DELIMITER '|';   -- 批量导入
SHOW TABLES;  DESCRIBE supplier;
PRAGMA buffer_stats;  PRAGMA extents(supplier);  PRAGMA log_level = debug;
```

`WHERE` 支持单个比较（`= <> < <= > >=`）。`COPY` 自动忽略 dbgen 每行末尾多出的 `|`，出错时报告文件名和行号。

4.4 元命令

元命令	作用
.tables、.schema t	列出表、显示表结构
.load t file	导入 .tbl 文件（COPY）
.get t p:s	按 RID 读取一行
.stats、.frames	缓冲池统计、每个帧的内容
.info t	表的文件、区、页、行数、填充率
.bitmap t、.extents t	区位图、区列表与页掩码
.fsm t、.free t	FSM 直方图、每页空闲空间
.page t p	页头与槽目录
.flush、.log [level]	刷盘、设置日志级别
.read f、.echo on|off、.timer on|off、.help、.quit	客户端功能

4.5 导入 TPC-H 数据与演示

把测试数据中的 .tbl 文件放到 test/tpc-h/ 目录，然后：

```
# supplier 验收演示：64 帧缓冲池，展示缓冲区淘汰和空闲空间的分配回收
rm -rf /tmp/demo_db
./build/minidb_shell --db /tmp/demo_db --buffer 64 --file scripts/demo_supplier.txt

# 导入全部 8 张表并逐行校验
./build/minidb_shell --db /tmp/tpch_db --file scripts/load_tpch.txt
./build/minidb_verify_tpch /tmp/tpch_db test/tpc-h
```

在嵌入式程序中使用时，链接 libminidb 并包含头文件 minidb/minidb.h，用法见 examples/demo.cpp。

5 测试方案和结果

5.1 测试方案

表 5: 测试层次

测试	内容
存储层测试 `minidb_storage_tests`	直接调用内部接口：CRC32C 硬件与查表实现一致；宽 CHAR 列的 schema 长度检查；转发后扫描不重复不遗漏、再次更新仍为一跳；页内增长不产生转发；FSM 回绕复用 hint 之前的空间；整区回收后新区填入空洞；FSM 页被清零后重新打开仍可用；4 帧缓冲池在大量淘汰下读写正确
集成测试 `minidb_tests`	通过 C API、8 帧缓冲池：执行实验文档 Listing 1 的 DDL；各类型插入查询往返；UPDATE/DELETE；2 万行插入并对 500 行做变长更新触发转发；COPY 导入与错误行号；rid 伪列；各 PRAGMA 结果；关闭后重新打开再比对；DROP TABLE 删除文件
验收演示 `demo_supplier.txt`	导入 supplier，展示区位图、区列表、FSM、槽目录、缓冲区淘汰日志，演示增删改
规模测试	导入 TPC-H SF1 全部 8 张表，重新打开后用 `minidb_verify_tpch` 与原文逐行比对

单元测试和集成测试在 `dev` 预设下开启 AddressSanitizer 和 UndefinedBehaviorSanitizer。

5.2 自动化测试结果

```
$ ctest --test-dir build --output-on-failure
1/2 Test #1: minidb_integration ..............   Passed    0.47 sec
2/2 Test #2: minidb_storage ..................   Passed    0.36 sec
100% tests passed out of 2
```

5.3 supplier 演示结果

使用课程提供的 `supplier.tbl`（10000 行, 1.4 MB），在 macOS（Apple M4 Pro）上以 Release 构建、64 帧（512 KB）缓冲池运行演示脚本：

```
rm -rf /tmp/demo_db
./build/minidb_shell --db /tmp/demo_db --buffer 64 --file scripts/demo_supplier.txt
```

1. 批量导入 10000 行在 0.014 s 内导入，页数从 1 增长到 213（1 个段头页 + 212 个堆页），文件扩展到 5 个区。64 帧的缓冲池在导入过程中发生 185 次淘汰，被淘汰的都是写满的脏页，因此写回 185 次；212 个新堆页直接在帧中初始化，不读磁盘：

```
COPY 10000 row(s) into supplier in 0.014 s (701245 rows/s); pages 1 -> 213,
  file 5 extent(s); buffer hit=30249 miss=5 evict=185 flush=185 new_page=212 hit_rate=99.98%
```

2. 三层空间结构 区位图中区 0–4 已分配（区 0 为系统保留）；段的区列表包含 4 个区，最后一个区用了 21 页。页填充率 98.73%，212 个堆页中 211 页的空闲空间小于 1 KB：

```
first_extent | last_extent | allocated | bits  | alloc_hint
0            | 4           | 5         | 11111 | 5

slot | ext  | first | used | page_mask
0    | 1    | 64    | 64   | H11111111111111111111111111111111111111111111111111111111111111111
1    | 2    | 128   | 64   | 11111111111111111111111111111111111111111111111111111111111111111
2    | 3    | 192   | 64   | 11111111111111111111111111111111111111111111111111111111111111111
3    | 4    | 256   | 21   | 1111111111111111111.....................................
```

第一个堆页（页 65）有 48 个槽，`free_lower` = 236、`free_upper` = 281，只剩 45 字节空闲。按 supplier 平均记录约 170 字节（含 18 字节记录头）加 4 字节槽估算，每页约 45–47 条，与实测一致。

3. 缓冲区换入换出 缓冲池只有 4 帧时导入 supplier 的前 150 行，日志完整展示了“淘汰脏页 → 写回 → 读入/新建”的过程，以及页的分配和 FSM 类别随插入逐步下降：

```
[DEBUG][BUF] EVICT frame=3 file=sys_tables.fsm pid=1 dirty=1
[DEBUG][BUF] FLUSH file=sys_tables.fsm pid=1
[DEBUG][BUF] MISS file=supplier.dat pid=64 -> frame=3
[DEBUG][SPACE] ALLOC file=supplier.dat page=65 seg=3
[DEBUG][BUF] EVICT frame=0 file=sys_tables.dat pid=65 dirty=1
[DEBUG][BUF] FLUSH file=sys_tables.dat pid=65
[DEBUG][BUF] NEW file=supplier.dat pid=65 type=6 -> frame=0
[DEBUG][BUF] EVICT frame=1 file=sys_columns.fsm pid=1 dirty=1
[DEBUG][BUF] FLUSH file=sys_columns.fsm pid=1
[DEBUG][BUF] MISS file=supplier.fsm pid=1 -> frame=1
[DEBUG][FSM] file=supplier.fsm spno=1 cat 0 -> 250
[DEBUG][HEAP] INSERT rid=(65,0) len=142 new page
[DEBUG][FSM] file=supplier.fsm spno=1 cat 250 -> 245
[DEBUG][HEAP] INSERT rid=(65,1) len=139
```

4. 更新与转发指针 把第 1 行（RID 65:0）的地址和注释改长。页 65 只剩 45 字节，放不下变长后的记录，于是新版本被插入到页 276 的槽 17，原槽 0 变为 6 字节的 `FORWARD` 指针，旧记录的 136 字节计入碎片：

```
更新前  slot | offset | length | flags   | forward_to
        0    | 8050   | 142    | USED    | NULL
更新后  0    | 8050   | 6      | FORWARD | 276:17
        page 65: frag_bytes 0 -> 136, free_space 45 -> 181; table_info: forwarded = 1
```

刷盘后在 DEBUG 级别用原 RID 读取，CLOCK 选出一个干净的帧淘汰（dirty=0，不需要写回），再从磁盘读入页 65，跟随转发指针后返回的是新内容，RID 仍为 65:0：

```
[DEBUG][BUF] EVICT frame=0 file=supplier.dat pid=216 dirty=0
[DEBUG][BUF] MISS file=supplier.dat pid=65 -> frame=0
rid  | s_suppkey | s_name            | s_address                                | ...
65:0 | 1         | Supplier#000000001 | A longer address: 40 characters long.... | ...
```

5. 删除与空间回收 删除前 3000 行后，行数 7000，页数从 213 降到 153，空出的整页被回收（区列表中的 .）。被转发的第 1 行删除时，目标记录和转发槽一起删除，forwarded 回到 0：

```
slot | ext  | first | used | page_mask
0    | 1    | 64    | 4    | H1.....1.1.....................................................
1    | 2    | 128   | 64   | 1111111111111111111111111111111111111111111111111111111111111111
```

随后在 DEBUG 级别插入一行，FSM 把它引导到已有空闲空间的页 128（spno=64，FSM 类别 102，即约 3.2 KB 空闲），页数保持 153 不增长：

```
[DEBUG][BUF] EVICT frame=51 file=supplier.dat pid=214 dirty=0
[DEBUG][BUF] MISS file=supplier.dat pid=128 -> frame=51
[DEBUG][FSM] file=supplier.fsm spno=64 cat 102 -> 98
[DEBUG][HEAP] INSERT rid=(128,0) len=115
```

6. 整区回收与复用 清空整张表后，除段的第一个区（段头页 + 第一个堆页）外，区全部还给区位图（11111 → 11000），区列表留下空洞；重新导入后空洞被依次填回，区号复用，文件没有增长：

```
slot | ext  | first | used | page_mask
0    | 1    | 64    | 2    | H1.........................................................
1    | NULL | NULL  | 0    | (hole)
2    | NULL | NULL  | 0    | (hole)
3    | NULL | NULL  | 0    | (hole)
bits = 11000, alloc_hint = 2
          --- 重新导入后 ---
COPY 10000 row(s) ... pages 2 -> 213, file 5 extent(s)
bits = 11111, alloc_hint = 5, 区列表恢复为区 1–4
```

7. 缓冲池统计与持久化 整个演示结束时缓冲池共命中 91416 次、未命中 1638 次（命中率 98.24%），淘汰 1808 次；.flush 写回全部 64 个脏页后脏页数为 0。关闭后重新打开数据库，table_info 仍为 10000 行、213 页，s_suppkey = 10000 的行仍在 RID 276:16。

5.4 规模测试：TPC-H SF1

使用课程提供的 benchmark_data.zip（SF1，8 张表共 8,661,245 行，约 1.0 GB），在 macOS（Apple M4 Pro）上以 Release 构建、默认 1024 帧（8 MB）缓冲池执行：

```
rm -rf /tmp/tpch_db
./build/minidb_shell --db /tmp/tpch_db --log-level info --file scripts/load_tpch.txt
./build/minidb_verify_tpch /tmp/tpch_db test/tpc-h
```

表 6: TPC-H SF1 导入结果（堆页数、区数取自导入后的 `table_info`）

表	行数	堆页数	区数	填充率	导入耗时	吞吐（行/s）
region	5	1	1	7.4%	<1 ms	—
nation	25	1	1	40.6%	<1 ms	—
supplier	10,000	212	4	98.7%	0.016 s	62 万
customer	150,000	3,448	54	99.1%	0.197 s	76 万
part	200,000	3,917	62	98.9%	0.221 s	90 万
partsupp	800,000	16,676	261	99.3%	0.895 s	89 万
orders	1,500,000	24,038	376	99.2%	1.581 s	95 万
lineitem	6,001,215	111,262	1,739	99.0%	8.281 s	72 万
合计	8,661,245	159,555	2,498		11.19 s	

- 8 张表的 `COPY` 合计 11.19 s；加上建表、统计输出和关闭时刷盘，整个进程耗时 14.0 s；
- 进程最大常驻内存约 15.4 MB（其中缓冲池 8 MB），数据库目录 1.2 GB。内存不随数据量增长，说明缓冲池在远小于数据量的情况下正常完成了内外存交换：仅 lineitem 一张表的导入就发生了 111,281 次淘汰和写回；
- lineitem 的区列表有 1739 项，超过段头页内联的 498 项，其余 1241 项放在 3 个溢出页中（段内共 111,266 页 = 1 个段头页 + 3 个溢出页 + 111,262 个堆页），验证了区列表溢出页的写入、重新打开和扫描；
- 重新打开数据库后用 `minidb_verify_tpch` 把每张表扫描出的每一行与 `.tbl` 原文逐行比对，8 张表全部一致：

```
region    rows=5 file=5 MATCH
nation    rows=25 file=25 MATCH
supplier  rows=10000 file=10000 MATCH
customer  rows=150000 file=150000 MATCH
part      rows=200000 file=200000 MATCH
partsupp  rows=800000 file=800000 MATCH
orders    rows=1500000 file=1500000 MATCH
lineitem  rows=6001215 file=6001215 MATCH
```

5.5 测试中发现的问题与重点覆盖的边界情况

- **字符串末尾空格丢失（已修复）**：逐行比对时发现输出函数会截掉所有字符串末尾的空格，导致 dbgen 注释字段末尾的空格在输出时丢失（存储本身正确）。修复后 VARCHAR 原样输出，CHAR 去掉填充空格，比较仍按 PAD SPACE 语义；
- **FSM 回绕**：只有一个 FSM 页的小表，如果搜索不回绕扫描 hint 之前的部分，删除释放的空间永远不会被复用。存储层测试专门验证 hint 之前释放的空间会被复用、不分配新页；

- **读盘失败的帧**：未命中时如果读盘失败，帧必须放回空闲链表，否则它既不在页表中也不在空闲链表中，缓冲池会逐渐变小。存储层测试通过读取超出文件末尾的页触发读盘失败来验证这一点；
- **macOS 编译失败（已修复）**：`fallocate` 是 Linux 专有接口，改为仅在 Linux 上使用，其他平台回退到 `ftruncate`。

6 实验总结

6.1 完成情况

实验要求	完成情况
段页式组织	页（8 KB）→ 区（64 页）→ 段；文件头、区位图、段头页、区列表溢出页、堆页
基本数据类型	char、varchar、int、long、float、double、date 全部支持，另支持 DECIMAL
空闲空间管理	区位图（区级）、区列表 + 页掩码（页级）、FSM + 槽目录（字节级）三层的分配与回收
缓冲区管理	帧数组 + 哈希页表 + 空闲链表；CLOCK 淘汰；pin 计数、页锁存、RAII Guard；命中/淘汰/写回统计
数据增删改	INSERT / UPDATE / DELETE，记录变长与转发指针，RID 稳定
测试数据规模	TPC-H SF1 全部 8 张表 866 万行，11 s 导入，重新打开后逐行校验一致

6.2 收获与体会

- **先定磁盘格式再写代码**。实验开始前用一天时间写设计文档、冻结页格式，并把后续实验需要的 `page_lsn`、`txn_id`、`undo_ptr` 预留进去。后续实验只需补全行为，不需要改格式、迁移数据；
- **RAII 管理 pin 和锁存**。手写 pin/unpin 很容易在错误返回路径上漏掉 unpin，几次操作后缓冲池就会被占满。所有页访问都通过 PageGuard 后，这一类问题在结构上就不会出现；
- **可观测性对调试和验收都很重要**。把缓冲池、区位图、区列表、FSM 和槽目录都以 PRAGMA 结果集形式暴露后，空间泄漏、FSM 不准等问题可以直接看出来，验收演示也能复用同一套命令；
- **用真实规模的数据测试**。小数据下所有路径都正常，导入 SF1 后才覆盖到区列表溢出页，也通过逐行比对发现了字符串尾部空格的问题；
- **FSM 只做提示**。FSM 不要求与数据页严格一致，插入时发现不准就修正，这样 FSM 的修改不用写日志，恢复实验中可以直接重建。

6.3 不足与后续工作

- 只实现了 CLOCK 淘汰策略，LRU-2 尚未实现（Replacer 接口已预留），可在验收后与 CLOCK 对比顺序扫描下的命中率；
- 尚未实现随机增删改与内存模型的差分测试；
- 每条语句自动提交，不支持回滚；没有预写日志，崩溃后不保证一致性（正常关闭时数据完整）。这部分留给并发控制和故障恢复实验，接口和页面字段已经预留；
- 缓冲池在全局锁内做磁盘 I/O，单线程下最简单，并发实验时需要改为“先占位、释放全局锁、再做 I/O”；
- 后续实验接入点：索引实验新增 B+ 树（复用 Segment 和 BufferPool，每个索引一个 `.idx` 文件）；查询处理实验在 TableHeap 之上实现火山模型执行器；并发控制实验基于稳定的 RID 实现两阶段锁；恢复实验实现 LogManager 并利用 `page_lsn` 和 `clean_shutdown` 做 ARIES 恢复。

A 核心源代码

A.1 CLOCK 淘汰（`src/buffer/buffer_pool.cpp`）

```
std::optional<frame_id_t> ClockReplacer::Evict() {
  if (size_ == 0) return std::nullopt;
  const size_t n = referenced_.size();
  for (size_t scanned = 0; scanned < 2 * n; ++scanned) {
    const frame_id_t frame = static_cast<frame_id_t>(hand_);
    hand_ = (hand_ + 1) % n;
    if (!evictable_[frame]) continue;
    if (referenced_[frame]) {
      referenced_[frame] = false;
      continue;
    }
    evictable_[frame] = false;
    --size_;
    return frame;
  }
  return std::nullopt;
}
```

A.2 缓冲区查找与换入（`src/buffer/buffer_pool.cpp`，有删节）

```
Result<frame_id_t> BufferPool::AcquireFrameLocked(
    const std::shared_ptr<DiskManager>& disk, page_id_t page_id,
    bool read_from_disk) {
  const PageKey key{disk->file_id(), page_id};
  auto found = page_table_.find(key);
  if (found != page_table_.end()) {                // hit
```

```
    frame_id = found->second;
    frame = frames_[frame_id].get();
    if (frame->pin_count == 0) replacer_->SetEvictable(frame_id, false);
    ++frame->pin_count;
    replacer_->RecordAccess(frame_id);
    ++stats_.hit;
  } else {                                          // miss
    auto result = TakeFreeFrameLocked();            // free list, else evict
    if (!result.ok()) return result.status();
    frame_id = result.value();
    frame = frames_[frame_id].get();
    frame->disk = disk;
    frame->page_id = page_id;
    frame->pin_count = 1;
    frame->dirty = false;
    page_table_[key] = frame_id;
    replacer_->SetEvictable(frame_id, false);
    replacer_->RecordAccess(frame_id);
    ++stats_.miss;
    if (read_from_disk) {
      const Status status = disk->ReadPage(page_id, frame->data);
      if (!status.ok()) {
        // Give the frame back as a free frame; it holds no valid page.
        page_table_.erase(key);
        ...
        free_list_.push_back(frame_id);
        return status;
      }
    } else {
      std::memset(frame->data, 0, kPageSize);
    }
  }
  return frame_id;
}

Result<frame_id_t> BufferPool::TakeFreeFrameLocked() {
  if (!free_list_.empty()) { ... return free_list_.front(); }
  std::optional<frame_id_t> victim = replacer_->Evict();
  if (!victim.has_value()) {
    return Status(StatusCode::kNoSpace,
                  "buffer pool is full: all frames are pinned");
  }
  Frame& victim_frame = *frames_[*victim];
  if (victim_frame.dirty) {                         // write back before reuse
    const Status status = WriteBackLocked(*victim);
    if (!status.ok()) { ... return status; }
  }
  page_table_.erase(PageKey{victim_frame.disk->file_id(), victim_frame.page_id});
  ...
```

```
  ++stats_.evict;
  return *victim;
}
```

A.3 区分配（src/space/space_manager.cpp）

```
Result<extent_id_t> SpaceManager::AllocateExtent(Transaction* /*txn*/) {
  auto guard_result = buffer_pool_.FetchWrite(disk_, kBitmapPageId);
  if (!guard_result.ok()) return guard_result.status();
  WritePageGuard guard = guard_result.TakeValue();
  std::byte* page = guard.MutableData();
  std::byte* bitmap = page + kBitmapOffset;
  const size_t bit_count = BitmapBitCount();
  const size_t hint = ReadU32(page, kBitmapAllocHintOffset) % bit_count;
  extent_id_t extent = kInvalidExtentId;
  for (size_t i = 0; i < bit_count; ++i) {
    const size_t bit = (hint + i) % bit_count;
    if (!TestBit(bitmap, bit)) {
      extent = static_cast<extent_id_t>(bit);
      SetBit(bitmap, bit, true);
      WriteU32(page, kBitmapAllocHintOffset,
               static_cast<uint32_t>((bit + 1) % bit_count));
      break;
    }
  }
  if (extent == kInvalidExtentId) {
    return Status(StatusCode::kNoSpace, "extent bitmap is full");
  }
  guard = WritePageGuard();  // release before touching the file header
  if (extent >= disk_->header().file_extents) {    // grow the file
    const Status status = disk_->ExtendTo(extent + 1);
    if (!status.ok()) return status;
    ...
  }
  ...
}
```

A.4 页分配与回收（src/space/segment.cpp，有删节）

```
Result<page_id_t> Segment::AllocatePageInternal(Transaction* txn) {
  // Look for a free page starting where the last allocation succeeded.
  for (size_t i = 0; i < slot_total; ++i) {
    const size_t index = (alloc_hint_slot_ + i) % slot_total;
    if (extents_[index].valid() && extents_[index].used_mask != kFullMask) {
      slot = static_cast<uint32_t>(index);  found = true;  break;
    }
  }
  if (!found) {
```

```
    auto extent = space_.AllocateExtent(txn);
    if (!extent.ok()) return extent.status();
    ExtentEntry entry;
    entry.extent_id = extent.value();
    // Fill the first hole left by a freed extent before growing the list,
    // so spno values stay dense.
    auto hole = std::find_if(extents_.begin(), extents_.end(),
                             [](const ExtentEntry& e) { return !e.valid(); });
    if (hole != extents_.end()) { *hole = entry; slot = hole - extents_.begin(); }
    else { extents_.push_back(entry); slot = extents_.size() - 1; }
  }
  const uint64_t mask = extents_[slot].used_mask;
  const uint32_t bit = static_cast<uint32_t>(__builtin_ctzll(~mask));
  extents_[slot].used_mask |= (1ULL << bit);
  ++page_count_;
  alloc_hint_slot_ = slot;
  return extents_[slot].extent_id * kPagesPerExtent + bit;
}

Status Segment::FreePage(Transaction* txn, page_id_t page_id) {
  ...
  const Status fsm_status = fsm_.Update(spno, 0);
  if (!fsm_status.ok()) return fsm_status;
  entry.used_mask &= ~(1ULL << bit);
  --page_count_;
  buffer_pool_.DiscardPage(space_.disk_ptr(), page_id);
  if (entry.used_mask == 0 && slot.value() != 0) {   // whole extent is free
    const Status status = space_.FreeExtent(txn, entry.extent_id);
    if (!status.ok()) return status;
    entry.extent_id = kInvalidExtentId;               // leave a hole
  }
  return Persist(txn);
}
```

A.5 FSM 搜索（src/space/segment.cpp，有删节）

```
Result<page_id_t> Segment::FindPageWithSpace(uint32_t need_bytes) {
  uint32_t need_cat = (need_bytes + 31) / 32;
  ...
  // Scan from the hint to the end, then wrap once and finish with the part of
  // the start page that precedes the hint.
  for (size_t k = 0; k <= page_total; ++k) {
    const size_t page_index = (start_page + k) % page_total;
    uint32_t lo = 0, hi = FreeSpaceMap::kEntriesPerPage;
    if (k == 0) lo = start_offset;
    else if (k == page_total) { hi = start_offset; if (hi == 0) break; }
    if (fsm_.MaxCategoryHint(page_index) < need_cat) continue;  // skip page
    ReadPageGuard guard = fsm_.ReadPage(page_index).TakeValue();
    uint8_t seen_max = 0;
```

```
    for (uint32_t offset = lo; offset < hi; ++offset) {
      const uint8_t category = ReadU8(guard.Data(), kPageHeaderSize + offset);
      if (category > seen_max) seen_max = category;
      if (category < need_cat) continue;
      ...                                   // check the page is allocated
      insert_hint_spno_ = spno;
      return entry.extent_id * kPagesPerExtent + bit;
    }
    if (lo == 0 && hi == FreeSpaceMap::kEntriesPerPage) {
      // A full scan found nothing usable: remember the page's real maximum.
      fsm_.SetMaxCategoryHint(page_index, seen_max);
    }
  }
  return kInvalidPageId;   // caller allocates a new heap page
}
```

A.6 记录更新与转发指针（src/table/table_heap.cpp，有删节）

```
Result<RID> TableHeap::Update(Transaction* txn, RID rid, const Tuple& tuple) {
  auto resolved = ResolveForward(rid, &hops, &moved_in);
  const RID target = resolved.value();
  // Steps 1 and 2: overwrite, or grow within the page.
  {
    WritePageGuard guard = buffer_pool_.FetchWrite(..., target.page_id).TakeValue();
    std::byte* page = guard.MutableData();
    const Status status = HeapPage::Replace(page, target.slot, record.data(), length);
    if (status.ok()) { free_space = HeapPage::FreeSpace(page); done = true; }
    else if (status.code() != StatusCode::kNoSpace) return status;
  }
  if (done) { segment_->UpdateFreeSpace(target.page_id, free_space); return rid; }

  // Step 3: move the new version elsewhere and keep a one-hop forward at rid.
  auto inserted = InsertRecord(txn, record,
      static_cast<uint8_t>(HeapPage::kSlotUsed | HeapPage::kSlotMovedIn));
  const RID new_rid = inserted.value();
  if (!(target == rid)) {
    // rid -> target -> new_rid collapses to rid -> new_rid.
    const Status deleted = DeleteAt(txn, target);
    if (!deleted.ok()) return deleted;
  }
  {
    WritePageGuard guard = buffer_pool_.FetchWrite(..., rid.page_id).TakeValue();
    HeapPage::SetForward(guard.MutableData(), rid.slot, new_rid);
    ...
  }
  return rid;   // the RID never changes
}
```